DOCUMENT 06 / SMART UNIVERSITY DIGITAL CAMPUS

Implementation & Feasibility

A gated route from the existing prototype to an institutionally operated pilot and a responsible scale decision.

Feasibility verdict A controlled pilot is technically realistic with the existing Flutter/FastAPI foundation. Success hinges on named university data owners, reliable master records, staff training, privacy review, and production operations. The proposed dates and staffing are planning assumptions.

What exists and what must be added

Dimension	Current evidence	Pilot requirement
Application	Windows Flutter portals and FastAPI routes for core journeys.	Device-specific UAT for every selected platform; prioritized defect fixes.
Data	Programme/curriculum catalogue, demo records, registration and allocation workflows.	Registrar-approved source data, imports, identity matching, ownership rules.
Operations	Local/dev backend; Docker Compose example with PostgreSQL.	Managed hosting, HTTPS, backups, monitoring, support rota, recovery tests.
AI	Campus-context routing and external Gemini call; heuristic risk warnings.	Answer evaluation, safeguards, budget, human review, calibration study.
Payments	Invoices and manual recorded payments.	Actual gateway reconciliation only after provider and finance approval.

Recommended pilot boundary

Start with one academic unit, one approved programme set, and a cohort of roughly 100-500 students. Enable registration, attendance/timetable viewing, a small service-certificate menu, helpdesk, and parent linkage where consent and records exist. Add hostel or transport only where staff already maintain allocation data. This cohort is a planning range, not a tested system limit.

Proposed 16-week rollout

Weeks	Work and owner	Exit gate
1-2	Discovery: product lead, registrar, finance, service owners map journeys and authoritative sources.	Signed pilot scope, owners, data dictionary, baseline measures.
3-5	Data preparation: registrar and technical team validate programmes, students, parent links, rooms/routes.	Import trial, duplicate review, reconciliation sample approved.
6-8	Engineering hardening: backend, frontend, DevOps address authorization, deployment, observability.	Security cases pass; restore drill and performance baseline.
9-10	User acceptance: admin staff, students, parents complete task scripts and accessibility review.	Critical defects closed; service procedures signed.
11-14	Limited live pilot: service owners run supported cohort and monitor daily exceptions.	Weekly KPI and incident reviews; rollback route ready.
15-16	Evaluation: compare baselines, data accuracy, cost, satisfaction, and support demand.	Written go / improve / stop decision by sponsor.

Schedule dependency Institutional approvals, cleaned records, payment procurement, and calendar timing can extend the schedule. Do not interpret the 16 weeks as a delivery commitment.

People and decision rights

A plausible core team is one backend engineer, one Flutter engineer, half-time QA, quarter-time DevOps, and a project lead. Registrar, academic, finance, student-services, privacy, and security representatives need allocated review time. These are estimated roles, not confirmed staffing or budget.

- Registrar owns student identity, programme, enrolment, and parent relationship accuracy.
- Academic owners approve timetable, attendance, examination, and intervention records.
- Finance owns invoice rules, payment reconciliation, refunds, and financial reporting.
- Hostel, transport, and certificate officers own inventory, allocation, issuance, and exceptions.

Data and integration path

Load official records through a validated import with dry-run reporting, duplicate detection, and repeatable identifiers. Staff should resolve conflicts before a record becomes student-visible. Use separate demo and institutional datasets. Treat the provided curriculum workbook as demonstration input until each programme, semester, subject, and credit summary is signed off by its academic owner.

Flow	Minimum control	Acceptance evidence
Student registration	Unique institutional identifier; duplicate checks; account-to-student binding.	Cross-account negative tests and roster reconciliation.
Academic updates	Named source owner; dated import; validation of programme/semester links.	Sampled student views agree with approved source.
Services	Room/seat capacity; status transitions; issue and revocation records.	End-to-end allocation and certificate tests.
Parent access	Admin-created link or approved consent process; relationship review.	Parent sees only linked students.
Fees	Invoice amounts and recorded payments reconciled by finance.	Sample balance and exception ledger.

Acceptance and recovery

- Test role and record boundaries for student, parent, and admin on every sensitive endpoint.
- Test ordinary read latency and errors at an agreed pilot workload. A suggested target is p95 under 2 seconds for ordinary reads and under 1% server errors; benchmark before adoption.
- Separate slower AI and PDF operations from the ordinary-read target; set timeouts and clear failure messages.
- Run a backup restore into an isolated environment and compare a small set of academic, fee, and certificate records.
- Release first to staff, then a small cohort, then expand after daily issue review. Keep a documented application rollback and data recovery procedure.

Costs, risks, and scale decision

Prepare a local costed estimate covering engineering, source-data clean-up, hosting, backups, AI requests, support, training, security review, and contingency. SMS/email, payment-gateway fees, and integrations need separate provider quotes. The prototype alone does not establish a reliable total cost or return on investment.

Risk	Owner	Mitigation / trigger
Inaccurate source records	Registrar	Reject import on duplicates; signed sample reconciliation before launch.
Cross-account data exposure	Security + engineering	Authorization tests and critical-fix gate before pilot.
Staff workflow not adopted	Service leads	Training, daily exception review, named response times.
AI error or cost spike	Product + engineering	Grounded-answer checks, quotas, review queue, provider timeout.
Service interruption	DevOps	Monitoring, backups, restore drill, incident ownership.
Uncontrolled scope growth	Sponsor	Written pilot boundary and gate-based change control.

Go / no-go criteria

Proceed beyond the pilot only when critical privacy and authorization issues are closed, service owners accept their workflows, record samples reconcile, backup restoration succeeds, observed workload meets the agreed thresholds, and pilot measures improve without excluding a student group. Otherwise extend the pilot or narrow scope.

Supporting references

Project: **backend/deployment/README.md**, **backend/docker-compose.production.yml**, **backend/app**, and Flutter **lib** workflows.

Flutter platform support https://docs.flutter.dev/reference/supported-platforms

PostgreSQL backup guidance https://www.postgresql.org/docs/18/backup.html